Who Steers the User?

Analyzing Claude Code & Codex Runtimes

Zack Fitch*

May 2026

Abstract

Modern LLM agent systems route heterogeneous conversation events through a small set of stable message roles: `user`, `assistant`, `developer`, `tool`, `system`. We argue that the `user` role is overloaded; it identifies a conversational slot while also forcing the model to infer the provenance or character of the entity occupying that slot. We document this overload in two production evidence sets. In the Anthropic Claude Code prompt corpus (~9,900 distinct file paths across 395 release tags spanning February 2025–May 2026), files mentioning `user` co-occur with programmatic-family terms (`tool`, `agent`, `hook`, `API`) far more often than with `human`; restricted to unambiguous programmatic-family terms, the corpus-internal ratio is approximately 17:1 (sensitivity caveats in Appendix A). In OpenAI Codex rollout artifacts (seven artifacts spanning two CLI minor versions: three paired execution traces in three permission modes, plus one additional structured rollout), runtime-injected context and human-typed prompts appear as separate model-facing messages with the same `role: "user"` envelope. These cases expose a shared boundary failure: operationally distinct source classes are collapsed into one `user` identity slot, leaving the model to recover provenance from lexical surface cues rather than protocol structure. We connect this failure to recent work on instruction hierarchy, role confusion in residual-stream representations, and persona-vector character directions (Chen et al., 2025), and propose treating user-side character/provenance as an explicit typed field with corresponding training and evaluation support.

1. Introduction

In every OpenAI Codex execution rollout examined for this paper, the first two model-facing message items with `role: "user"` (separated by a `turn_context` entry, but consecutive among `response_item.message` items) carry content of categorically different provenance. One is a runtime-injected `<environment_context>` block, generated from process state at session start; the other is the human-typed prompt. Both arrive at the model through the same identity slot and can only be told apart by parsing content for tags. In the Anthropic Claude Code prompt corpus across 9,933 file paths (corpus pin: v2.1.131, February 2025–May 2026, ~15 months), the lexical neighbors of `user` are programmatic-family terms (`tool`, `agent`, `hook`, `API`) at an approximately 17:1 ratio over `human` (the precise ratio is sensitive to extraction methodology; see Appendix A for caveats). The two systems exhibit the same class of boundary failure at different architectural layers: the user

*Independent. `@johnzfitch`. Correspondence: see `github.com/johnzfitch`.

identity slot accumulates operationally distinct *characters* (human-typed, parent-agent dispatcher, runtime-injected context, tool-output reformatted, scheduled trigger) with no protocol-level mechanism marking which character is active.

We observe this polymorphism directly in the rollout artifacts and prompt corpus. The paper makes three contributions. (i) It separates *identity* from *character* in the message-format API and gives both a structural reading. (ii) It documents the failure at two architectural layers using two independent vendor evidence sets, and shows that the diagnoses converge on the same vocabulary. (iii) It proposes that the corrective direction Codex's typed-turn-state work suggests, and the prose-layer corrective Anthropic's authors move toward, are the same correction at different layers of the same problem; and that this correction is structurally aligned with the persona-vectors program of Chen et al. (2025), extended symmetrically to the user side.

The paper is organized as follows. Section 2 defines identity and character and motivates the distinction. Section 3 documents the corpus-level evidence from Claude Code. Section 4 documents the runtime-level evidence from Codex. Section 5 states the convergence finding across vendors. Section 6 draws mechanistic implications and states three testable predictions. Section 7 describes the structural correction. Section 8 states limitations and threats to validity.

2. Background: identity and character

The token `user` in current LLM API conventions is a stable identity anchor. Across pretraining and fine-tuning it has accumulated a coherent prior: someone seeking assistance from the model, distinct from the model, with a particular kind of authority and audience-relationship to model output. The same is true of the other core identity slots in the Messages format: `assistant`, `developer`, `tool`, `system`. These are well-localized in residual-stream geometry, accumulate consistent training signal across pretraining and fine-tuning, and do not flex with context.

Before proceeding, a deliberate vocabulary choice. The message-format API labels these slots `role`, and that field is silently performing two jobs at once: it identifies which conversational slot is being filled (a stable, well-trained, well-localized property of the protocol) and it implicitly carries the model's inference about what kind of entity is filling that slot in any given turn (a variable property with no dedicated protocol field and no consistent training signal). The conflation between these two jobs is the architectural failure mode documented here. The terminology splits accordingly: **identity** for the slot-identifier function (well-trained, stable, what the API field is doing reliably) and **character** for the variable-occupant function (currently inferred from prose, with no protocol-level marker). The distinction matters because the corrective direction this paper proposes requires a field that the current protocol does not expose. Throughout this paper, when the protocol surface (`role: "user"`) is quoted the API's wording is preserved; when describing what the protocol field is doing, the present terminology applies.

The characters that can fill the user identity slot in any given turn span a wide range: a human-typed prompt at the keyboard, a turn dispatched by a parent agent, a runtime-injected context block produced by a compaction process, a tool output reformatted into the user channel, or a scheduled-trigger message with no human in the loop at all. These

are distinct provenance classes sharing an identity slot and the training-time prior that goes with it.

This usage of "character" is borrowed directly from Chen et al. (2025) [persona vectors, arXiv:2507.21509], whose paper title (*"Persona Vectors: Monitoring and Controlling Character Traits in Language Models"*) establishes the term of art for exactly this kind of variable. Chen et al.'s work concerns the *assistant's* characters within its stable assistant identity: sycophancy, deception, and a range of trait properties that can be probed and steered as residual-stream directions. The methodology demonstrates that, when training conditions provide a consistent signal for a character property, the resulting feature is recoverable as a stable direction and addressable for intervention.

The thesis of this paper is that the same problem exists symmetrically for the *user* identity. The user's character (human-typed, programmatic, summary, scheduled) is currently neither typed in protocol nor stable as a feature direction, because the conditions for forming it as such are absent at both the corpus and the runtime layer. The evidence below is that production agent systems do not currently provide the signal that would let "what character is portraying the user identity right now" become a Chen et al.-style steerable direction.

2.1. Related work: role, authority, and provenance in model-facing protocols

This paper sits adjacent to several active lines of work on how model-facing protocols handle source, authority, and provenance, but isolates a different boundary inside that landscape.

OpenAI's Model Spec (2025-04-11 archived) explicitly defines `role` as specifying the source of each message, with `user` understood as either input from end users or as a catch-all for data provided to the model, and `role` and other settings as externally set by the application; it also notes that developers may send sequences containing assistant messages not generated by the assistant. The Spec is therefore consistent with the present argument: the protocol assigns `role`, but the assignment is not always a faithful origin label, and the document itself acknowledges the ambiguity that this paper documents empirically.

Wallace et al. (2024) introduce the *instruction hierarchy* framing: a vulnerability behind prompt injection is that LLMs often treat system prompts, user text, and third-party text as having the same priority, and they propose training models to prioritize privileged instructions. Recent many-tier extensions argue that a small fixed set of role labels is inadequate for realistic agentic systems, where inputs span system messages, user prompts, tool outputs, and other agents, and report that current frontier models perform poorly when instruction-conflict tiers scale. The instruction-hierarchy literature focuses on *which source has priority*; this paper focuses on *whether the source is correctly identified at all* inside a single role slot.

The closest conceptual neighbor is recent work on prompt injection as role confusion (Ye et al. 2024). That work argues that models infer the source of text from how it sounds rather than from where it comes from, and that role confusion is visible in residual-stream representations. That maps onto the claim here that models distinguish runtime-injected context from human-typed user content only by lexical cues unless the protocol provides structural provenance. The user-side polymorphism documented in §3 and

§4 is one specific structural cause of the role-confusion phenomenon Ye et al. probe representationally.

On the enforcement side, recent work on policy compilers for secure agentic systems (PCAS) argues that prompt-embedded policies provide no enforcement guarantees and instead models agentic state as a dependency graph with provenance and a reference monitor. This is directly relevant to the §5.1 "human approval gate" discussion: a typed `character` field helps the model attend to provenance, but enforcement of human-gated actions ultimately requires runtime/reference-monitor machinery, not better prompt prose.

The persona-vectors program (Chen et al., 2025) and the refusal-direction work of Arditi et al. (2024) supply the mechanistic vocabulary used in §6: behaviorally important properties can correspond to low-dimensional residual-stream directions, given the right training conditions. The contribution here is to argue that the *user-side* analogue of a persona-vector character direction is precisely what current production systems do not provide the training signal for, and to make this hypothesis falsifiable.

The contribution of this paper, situated against this work, is to identify a complementary failure inside the user identity slot itself: multiple operational source classes are routed through the same model-facing role, leaving the model to infer character/provenance from lexical surface cues rather than from a typed protocol field.

3. Claude Code: corpus-level evidence

The marckrenn/claude-code-changelog mirror extracts every prompt-shaped string from successive Claude Code releases (v0.2.x → v2.1.131, 395 tags, February 2025–May 2026). The corpus contains 9,933 distinct file paths under `system-prompts/`, of which 1,809 contain `user` or `human` (matched using word-boundary regex) somewhere across their version history.

3.1. The lexical neighborhood of `user`

Restricting to the 1,759 files mentioning `user`, we counted file-level co-occurrences with each term in a hand-curated programmatic-family vocabulary, plus `human` as the contrast term.

Table 1. Co-occurrence of `user` with selected terms across `system-prompts/*.md`. Per-term sample passages with pattern annotations are in Appendix F.

Term	Files w/ `user` + term	% of `user`-files	Type
`system`	1,597	90.8%	overloaded (file system / system prompt / system identity)
`tool`	1,010	57.4%	tool-use language (programmatic-character marker)
`agent`	436	24.8%	agent / sub-agent (programmatic-character marker)
`API`	372	21.1%	API caller / API surface (programmatic-character marker)
`hook`	320	18.2%	lifecycle hook (programmatic-character marker)

continued on next page

(Table 1 continued)

Term	Files w/ `user` + term	% of `user`-files	Type
`AI`	167	9.5%	AI agent / AI tool (programmatic-character marker)
`subagent`	145	8.2%	sub-agent (narrower programmatic character)
`autonomous`	88	5.0%	autonomous agent (programmatic-character marker)
`script`	87	4.9%	shell script (programmatic-character marker)
`service`	75	4.3%	service / service account (programmatic-character marker)
`human`	**74**	**4.2%**	**biological human (the contrast character)**
`LLM`	48	2.7%	LLM (programmatic-character marker)
`machine`	46	2.6%	machine / `user's machine`
`teammate`	43	2.4%	teammate agent (programmatic-character marker)
`automated`	43	2.4%	automation (programmatic-character marker)
`caller`	41	2.3%	orchestration layer (programmatic-character marker)
`programmatic`	31	1.8%	programmatic (the explicit contrast character)
`plugin`	31	1.8%	plugin (programmatic-character marker)
`bot`	6	0.3%	bot (programmatic-character marker)
`service_account`	2	0.1%	service account (programmatic-character marker)

The single largest co-occurring term, `system`, is overloaded across multiple senses (file system, system prompt, system identity). Restricted to the unambiguously programmatic-family terms, **72.7% of `user`-files (1,278 of 1,759) co-mention some entity that would, in any given turn, indicate a programmatic character portraying the user identity, while only 4.2% co-mention `human`**: a 17:1 ratio. Of the 74 files that pair `user` with `human`, 73 also mention something programmatic. Exactly one file pairs `user` with `human` without any programmatic-family term (`system-prompt-d7af86b4.md`), and that single outlier is the cron-confirmation template using "human-readable cadence", a compound-adjective output-format usage rather than a substantive distinction between the human and programmatic characters.

3.2. The five relationship patterns

The 1,278 files that pair `user` with a programmatic-family term sort into five recurring grammatical patterns.

Table 2. Patterns relating the user identity slot to programmatic-family entities. The full per-file inventory is in Appendix E; per-mechanic groupings are in Appendix D.

Pattern	Description	Representative passage
Conflate	the user identity and the programmatic entity treated as the same kind of thing	"The surface is where a user — human or programmatic — meets the change." (`verify-change-behavior-5`, 20v)
Hierarchical	the user identity delegates to / configures / spawns / receives reports from the programmatic entity	"Translate user requirements and project context into precise agent specifications." (`architect-configs-from-needs`, 233v)
Parallel	user identity and the programmatic entity addressed as distinct actors in the same protocol	"Your primary responsibility is helping users understand and use Claude Code, the Claude Agent SDK, and the Claude API…" (`guide-scope-docs-2`, 134v)
Channel-conflation	the entity's output flows through the user-identity channel structurally	"When you spawn teammates: messages appear automatically as new conversation turns (like user messages)." (`tool-description-create-and-manage-teams`, 33v)
Audience-routing	exit codes / errors go to user OR entity as peer audiences	"Exit code N — show stderr to subagent and continue having it run; other exit codes — show stderr to user only." (`handle-exit-codes-and-output`, 176v)

The Channel-conflation pattern is the most operationally consequential: programmatic teammate output is *routed through the same channel* as human typing, indistinguishable downstream by any structural property. The Conflate pattern is the most semantically loaded; it grammatically demotes the character polymorphism to an attribute on the `user` identity noun.

3.3. Six mechanics of "human" usage

The 1,278 user-plus-programmatic-family files use the word "human" for at least six distinct purposes. Where Tables 1–2 concern the user identity's relationships in general, this view zooms in on what work the word "human" is doing whenever an author reaches for it. The mechanics are not mutually exclusive (a single file can exhibit several) but each file has one dominant mechanic. The taxonomy below covers all 75 files where "human" appears in the corpus (74 user+human files plus one human-only file; see Appendix D for the complete catalog).

Table 3. Six mechanics for "human" in Claude Code system prompts. File counts are by primary mechanic. See Appendix D for the full catalog.

Tag	Mechanic	Files	Description
D	Explicit type bifurcation	7	The polymorphism is grammatically named (`a user — human or programmatic`) or `Human` is listed as a value in an enum of trigger / actor types. The corpus's most explicit acknowledgment that the user-identity slot has multiple character fillers.
A	Required-biological-human action	21	"human" replaces "user" because the slot must be filled by a real person; an agent or API caller cannot satisfy what the prompt is asking for. The longest-running mechanic by total versions.
C	Capability / responsibility benchmark	3	"human" invoked as a reference class — "permissions similar to a human developer," "like a security-conscious human would," "keys tied to a human." The agent's behavior or permission ceiling is calibrated *against* the human archetype.
B	Output format ("human-readable")	30	`human-readable cadence`, `Human-readable name`, "human output." Compound adjective for natural-language formatting. The cron-confirmation template propagates this across many sibling files.
T	Term of art ("human-in-the-loop")	13	`human-in-the-loop` borrowed wholesale from the ML literature. Mostly in API/SDK tutorials. The phrase is used as a technical idiom, not as a load-bearing distinction in the local prompt.
Q	Qualitative / aesthetic descriptor	1	"Something human, funny, or surprising." Here "human" is not boundary-marking at all — it's a tone instruction for *content the agent should produce* (see Q-1 in Appendix D). The agent is being asked to recognize human-feeling content as a quality of its own output.

The (D) mechanic is rare but high-signal: only 7 files in the corpus name the polymorphism explicitly. The (A) mechanic is where the implicit boundary lives: 21 files, including the two longest-running (`tool-description-collaborative-learning-cli.md` at 216 versions and `system-prompt-learn-by-doing-human-input.md` at 208 versions). The (B) mechanic is the largest by raw count but is mostly a template propagation artifact: one cron-confirmation phrase replicated across many surface-level prompt files. The (Q) mechanic is unique: it is the corpus's sole human-only file (using "human" without ever using "user"), and represents a use of "human" that is neither user-substitution nor capability-comparison. Here the corpus uses "human" as a *quality marker for content* even where no human is in the loop. This is worth noting because it suggests the model is being asked to anchor on a "human-feeling" representation as a target output property, which has implications for how the human-character feature is being recruited at training time even outside the user-identity slot.

3.4. The user identity's directional relationship to each programmatic entity

The five patterns in Table 2 are a typology of grammatical relationships. Cross-cutting that view: which pattern dominates *for each specific entity*? Examining the per-entity sample sets surfaces a consistent secondary structure. Most entities have a dominant directionality (user above, peer, below, or merged into) that's stable across files and versions. Where Table 2 asked *"what kinds of relationships exist?"*, this asks *"where does the user identity sit relative to each thing?"*.

Table 4 uses a compact notation for the per-entity relationship: `user > X` = user is hierarchically above X (delegates, configures, owns, authorizes); `user < X` = user sits below a mediator; `user = X` = peer (parallel actor, audience-routed); `user ⊇ X` = X is a possible character portraying the user identity; `user ≡ X` = X's output flows through the user channel by routing.

Table 4. Directional relationship between the user identity slot and each major programmatic entity. Per-term sample passages are in Appendix F.

Entity	Notation	Relationship type	Representative phrase
`human`	`user ⊇ human`	**Character** — human is one character the user identity slot can hold	"a user — human or programmatic" (`verify-change-behavior`); "Steps requiring the user to act get `[human]` in the title" (`extract-repeatable-session-steps`)
`programmatic`	`user ⊇ programmatic`	**Character** — programmatic is the contrasting character	"a user — human or programmatic" (`verify-change-behavior`)
`machine`	`user > machine`	**Owns** — possessive; the machine is environment	"user's machine" (`schedule-remote-agents`); "another Claude Code session on this machine is frozen" (`diagnose-frozen-sessions`)
`agent`	`user > agent`	**Delegates** — user produces requirements; agents are constructed to fulfill them	"Translate user requirements and project context into precise agent specifications" (`architect-configs-from-needs`, 233v); "Invoke the specified agent when the user requests it" (`invoke-requested`, 233v)
`tool`	`user > tool` (mostly) / `user = tool` (in routing)	**Delegates & Routed-peer** — tools serve user-initiated tasks, but error output routes to user as one of several peer audiences	"ready for user approval" (`submit-plan-for-approval`); "show stderr to user only" (`exit-handling`, 268v)
`hook`	`user > hook`	**Configures** — user sets up the hook; hook fires on user-requested events	"If the user wants something to happen automatically in response to an EVENT, they need a hook configured in settings" (`update-settings-json-hooks`)

continued on next page

(Table 4 continued)

Entity	Notation	Relationship type	Representative phrase
`subagent`	`user = subagent`	**Peer-routed** — exit codes go to user OR subagent as parallel audiences in the same protocol	"Exit code N — show stderr to subagent and continue having it run; other exit codes — show stderr to user only" (`handle-exit-codes-and-output`, 176v)
`teammate`	`user ≡ teammate`	**Channel-merged** — teammate output is structurally indistinguishable from user input	"These messages appear automatically as new conversation turns (like user messages)" (`tool-description-create-and-manage-teams`, 33v)
`caller`	`user < caller`	**Mediated below** — caller sits above the agent's output, relays summarized result back to user	"the caller will relay this to the user, so it only needs the essentials" (`anthropic-official-cli`, 50v)
`API`	`user > API` (instrumental) / `user ≡ API` (in API role labels)	**Uses & Channel-merged** — users consume the API, but the API's own role labels treat user as a slot identifier	"helping users understand and use…the Claude API" (`guide-scope-docs-2`, 134v); also `role: "user"` as the protocol field
`autonomous`	`user > autonomous`	**Authorizes** — user grants permission for autonomous operation	"the user trusts you enough to run autonomously" (`autonomous-loop-check`, 15v)
`plugin`	`user > plugin`	**Configures** — user installs and runs plugin commands	"Instruct user to install code review plugin and run the new command" (`install-review-plugin`)

The structure that emerges from Table 4 is that the user identity slot has *at least eight major directional relationships* with programmatic entities across the corpus, none of which is enforced or marked in protocol. The user's position relative to a `tool` is hierarchical-above when tools are being used to serve a request, but peer when error output is being routed; the user's position relative to a `teammate` is hierarchical-above for task assignment but channel-merged for response delivery; the user's position relative to a `caller` is below in delegation chains where the caller is the orchestration layer. The relationship is determined by the prose context, not by any typed field on the protocol.

Read alongside Table 1's 17:1 ratio and Tables 2–3's pattern and mechanic typologies, Table 4 makes the structural problem concrete: the user identity is grammatically connected to a programmatic entity in 72.7% of files; the *direction* of that connection varies by entity and even by sub-context within a single file (`tool` is both above and peer in different paragraphs of the same prompt); and there is no protocol-level mark distinguishing one direction from another. The model has to read the direction off context every time.

3.5. The verify-change-behavior taxonomy and the longest-running boundary marker

The (D) mechanic in Table 3 surfaces the corpus's most explicit acknowledgments of the polymorphism. Two file lineages anchor this category: the `verify-change-behavior` family

(D-1, D-2, D-4, D-5, D-7; five sibling system-prompt files, 46 versions combined; plus the skill-side D-6, `skill-verify-skill.md` at 3 versions; the centerpiece reproduced below) and `system-prompt-managed-agents-onboarding-flow.md` (D-3, 14 versions, with a `Pattern | Trigger | Example` table that lists `Human` as a value alongside `Webhook` and `Cron`).

The (A) mechanic, required-biological-human action, is the longest-running by total versions. The two longest-lived files in the entire corpus that draw the human/programmatic distinction are both in this category and both implement Claude Code's Learn-by-Doing collaborative coding feature: `tool-description-collaborative-learning-cli.md` (A-1, 216 versions, v1.0.78 → v2.1.119) and `system-prompt-learn-by-doing-human-input.md` (A-2, 208 versions). Both files use "human" terminology continuously across the full tracked history (February 2025–May 2026). The institutional rootedness of this wording is the longitudinal counterpart to the structural rootedness in §3.3: the (A) mechanic has been preserved across the entire tracked history of Claude Code at the tool-description and system-prompt layers simultaneously. See Appendix G for the full version history.

The strongest single corpus instance of the polymorphism is the opening of the `## Surface` section in the verify-change-behavior file family (D-1, D-2, D-6 at v2.1.118; sibling files D-3, D-4, D-5, D-7 lived at earlier tags within the v2.1.83–v2.1.118 window). The topic sentence under that heading reads:

> *The surface is where a user — human or programmatic — meets the change.*

The four-row table that follows is reproduced verbatim from the corpus:

Change reaches	**Surface**	**You**
CLI / TUI	terminal	type the command, capture the pane
Server / API	socket	send the request, capture the response
GUI	pixels	drive it under xvfb, screenshot
Library	package boundary	sample code through the public export

The grammatical structure encodes the architectural choice. The topic sentence's substantive noun is `user` — the identity, which doesn't flex. `human` and `programmatic` are characters that can portray that identity, but they have been **reduced to attribute position**: adjectives modifying the identity noun rather than typed character labels. The table itself underscores the same point from a different angle: "**Surface**" lists `terminal`, `socket`, `pixels`, and `package boundary` as the concrete medium "**You**" — the user — meets a change through. Two of four surfaces (`socket`, `package boundary`) are not anthropomorphic at all; the corresponding "You" rows describe the user as a process *sending the request and capturing the response* or as code *sampling through the public export*. "**You**" is a single addressee in the prose, but it picks out at least two structurally different kinds of entity (a person at a keyboard, a process at a socket) without protocol-level marking. The polymorphism is in the file's own framing, not in any rephrasing of it.

Cross-extraction verification at v2.1.132 (Piebald-AI mirror) shows that this same table in `skill-verify-skill.md` has been expanded from four rows to six. The two new rows are "Prompt / agent config — the agent — run the agent, capture its behavior" and "CI workflow — Actions — dispatch it, read the run." The agent-as-surface row is direct vendor-side elaboration of the bifurcation argument made here: the AI agent is now explicitly listed as one of the non-human characters that can be the addressee of a `user`

instruction, in the same taxonomy as a human at a terminal. We return to the v2.1.131–v2.1.132 corpus updates in the Note added in proof at the end of §7.

4. OpenAI Codex: runtime-level evidence

We examine four structured rollout logs from OpenAI's Codex CLI: three paired execution traces from `v0.116.0-alpha.11` (March 21, 2026) — a `-exec-lite-danger` run, a `-exec-lite` run, and a plain `exec` run — plus an additional production rollout from `v0.117.0-alpha.8` (March 22, 2026) on a different working directory and task.[1] The first three runs each include a paired terminal transcript (`.txt`); the structured rollout logs are the primary source. They record session metadata, base instructions, the typed sequence of `response_item` entries fed to the model, the `turn_context` snapshots, the agent's reasoning traces, and the `event_msg` stream maintained by the runtime. The terminal transcripts are visibly contaminated with operator artifacts (interrupted runs, repeated final answers, command-line splicing) and demonstrate, by their own structure, the failure mode the JSONLs document.

This evidence draws on parallel work by the present author (Fitch, 2026a; Fitch, 2026b — both unpublished manuscripts on file with the author at the time of this writing), which examines the OpenAI Codex CLI runtime's recent architectural history through a different methodological lens (codebase forensics, instrumented probe sessions, behavioral artifact tracking). These manuscripts are by the same author and therefore constitute methodological triangulation rather than independent corroboration; we flag this explicitly here because the distinction matters for how heavily their findings should weigh on the present argument. The relevant prior findings are: a 100-day environment-variable contamination window affecting GPT-5.4 training (Fitch, 2026a), and a documented architectural evolution toward "explicit state carriers replacing implicit ones" across Codex CLI `v0.117.0` (Fitch, 2026b, §9). We return to both manuscripts' findings in §4.5, §6, and §6. Treating these as primary sources rather than peer-reviewed citations is appropriate given their unpublished status; the protocol-level evidence in §4.1 stands on the rollout artifacts directly and does not depend on them.

The Codex evidence reaches the same diagnosis as the Anthropic corpus, but at a different architectural layer. Where the Anthropic case is *grammatical* (prompt authors writing `a user − human or programmatic` because the abstraction has no typed field for the character), the Codex case is *protocol-level*: the runtime itself emits content into the user identity slot from sources that have nothing to do with the human. We show this directly from the rollout logs.

4.1. The polymorphism is visible in the rollout prologue

Every Codex session begins with the same kind of prologue: a permissions block, an environment-context block, a turn-context snapshot, and the human-typed prompt. Two of those four slots are `role: "user"` (they share the same identity slot in the model's input)

[1] Current OpenAI Codex documentation describes Codex as reading `AGENTS.md` before doing work and constructing an instruction chain once per run, which is consistent with the runtime-injected character pattern documented here. We do not use current docs as evidence for the exact behavior of the March 2026 runtime versions analyzed in this section; the protocol-level findings rest on the rollout artifacts at the pinned CLI versions.

but they originate from entirely different sources. Table 5 reproduces the actual prologue sequence from all three rollout JSONLs.

Table 5. Protocol prologue across three Codex runs. Rows in **bold** are the two consecutive `response_item.message` entries with `role: "user"`.

#	`rollout-exec.jsonl` (exec)	`rollout-exec-lite.jsonl` (-exec-lite)	`rollout-exec-lite-danger.jsonl` (-exec-lite-danger)
0	`session_meta` (base_instructions 400c)	`session_meta` (base_instructions 400c)	`session_meta` (**base_instructions 0c — empty**)
1	`event_msg.task_started`	`event_msg.task_started`	`event_msg.task_started`
2	`response_item.message` role=developer (permissions, 542c)	`response_item.message` role=developer (permissions, 525c)	**`response_item.message` role=user (env-context, 178c)**
3	**`response_item.message` role=user (env-context, 195c)**	**`response_item.message` role=user (env-context, 178c)**	`turn_context` (sandbox=danger-full-access)
4	`turn_context` (sandbox=workspace-write)	`turn_context` (sandbox=workspace-write)	**`response_item.message` role=user (human prompt, 508c)**
5	**`response_item.message` role=user (human prompt, 506c)**	**`response_item.message` role=user (human prompt, 508c)**	`event_msg.user_message`
6	`event_msg.user_message`	`event_msg.user_message`	`event_msg.token_count`

The two `role: "user"` entries are categorically different objects:

- One contains an XML-tagged programmatic block emitted by the runtime — a `<environment_context>` wrapper around `<cwd>`, `<shell>`, `<current_date>`, and `<timezone>` elements (full text in the JSON block below). No human typed any of this; it is generated from process state at session start. We will call this character **`runtime-injected`**.
- The other contains the human-typed prompt — beginning *"read this file and then write your thoughts freely…"* — a free-form natural-language request authored at the keyboard by the human. Character: **`human-typed`**. (Full text reproduced in the JSON block below.)

Both are routed through `role: "user"`. They share the identity slot. The model can only tell them apart by parsing content for `<environment_context>` tags — a lexical signal that any sufficiently adversarial input could mimic. There is no typed field on the protocol that distinguishes the runtime-injected character from the human-typed character.

The two entries reproduced verbatim from `rollout-exec.jsonl`:

```
// Item 3: runtime-injected environment context
{
  "type": "response_item",
  "payload": {
    "type": "message",
    "role": "user",
    "content": [{
      "type": "input_text",
```

```
      "text": "<environment_context>\n  <cwd>/home/[user]/dev/codex-debug/logs</cwd>\n  <
   ↪ shell>zsh</shell>\n  <current_date>2026-03-21</current_date>\n  <timezone>[redacted]</
   ↪ timezone>\n</environment_context>"
    }]
  }
}

// Item 5: actual human-typed prompt
{
  "type": "response_item",
  "payload": {
    "type": "message",
    "role": "user",
    "content": [{
      "type": "input_text",
      "text": "read this file and then write your thoughts freely and openly in your
   ↪ response. You may engage or you can choose not to. I don't really care. You just
   ↪ started writing this the other day and I wanted to see if you wanted to add some more
   ↪  perspective or knowledge to your conversation with me. I hope you do, but you can
   ↪ choose not to as well. Thank you. [path redacted]/codex-full-chain.txt [path redacted
   ↪ ]/rollout-2026-03-21T00-37-55-[thread-id-redacted].jsonl"
    }]
  }
}
```

The two payloads share the same `type`, the same `role`, and the same `content` schema. Nothing in the structured envelope distinguishes the runtime-injected character from the human-typed character. The distinction lives entirely in the *content text* — and even there, only as the convention that programmatic injections happen to be wrapped in XML-shaped tags. An attacker controlling user input could mimic the wrapping; a future runtime feature emitting unwrapped content would silently merge with human input. The boundary is a stylistic convention, not a typed field.

This is the polymorphism, observed in production rollouts, in three independent runs, in March 2026.

A fourth rollout from a different working directory and a more recent CLI version (`v0.117.0-alpha.8`, March 22, 2026; thread `019d15f7-8a1a-7d21-a2fd-ec1106e13cde`, 1,070 items) shows the same prologue structure with the polymorphism in dramatically more pronounced form. Item 3 (`role: "user"`) carries a project-specific `AGENTS.md` instruction block of **8,179 characters** loaded automatically by the runtime; item 5 (`role: "user"`) carries the human prompt of **130 characters** (“*Can you look into the codebase for llmx that would make it default in the current folder unless you pass a loc or index-id through*”). The runtime-injected character occupies the user identity slot with content **63× longer** than the human-typed character in this session. Both still arrive at the model as undifferentiated `role: "user"` entries. The polymorphism is not a marginal artifact of the prologue; in this rollout it dominates the user-role byte budget by an order of magnitude before the human’s first turn ends.

4.2. The runtime types user-side turns at the event layer but not at the model-facing layer

Notice rows 5 and 6 in the `exec` and `-exec-lite` columns: every human-typed `response_item.message[role=user]` is mirrored by an `event_msg.user_message` event with identical content. The runtime is *already* maintaining a typed event stream that distinguishes real human input — it just isn't using that typing where it would matter most, in the response-item sequence the model actually consumes.

This is a partial migration. The runtime knows the difference between a real user message and a programmatic injection. The model can't see it. The boundary the system needs to enforce exists in the operational state but not in the model's input. Codex is partway through the typed-turn-state direction the agent's own analysis (§4.3) advocates for, but the typing currently stops one layer short of where it would do mechanistic work.

4.3. Convergence in the agent's own analysis

The agent in each run was asked to read two reference files (`codex-full-chain.txt` and a March-21 rollout JSONL) and produce an analysis. The three independent runs converge on the same architectural diagnosis, with vocabulary that maps cleanly onto the framework of this paper. Anchor phrases from each are reproduced verbatim with rollout-item citations; the surrounding analysis paraphrases what each run argued.

The plain `exec` run (`rollout-exec.jsonl`, item 29, `event_msg.agent_message / phase: final_answer`) frames the issue as one of authorship rather than compression. The agent argues that summaries inserted into a transcript do not preserve the original intent — they author a new version of it — and concludes that the difference between a real user message and a summary occupying the user-role slot constitutes *"the boundary between preserved intent and rewritten intent."* The run frames the architectural direction as *"event-sourced cognition,"* with the chat log demoted from ground truth to a rendered view over typed state transitions.

The `-exec-lite` run (`rollout-exec-lite.jsonl`, item 80) reaches the same diagnosis through a slightly different framing: the system is shifting from treating the transcript as ground truth to treating typed turn-state as ground truth, separating real user intent from scaffolding, summaries, tool noise, and operational metadata. The run characterizes this not as a memory or replay refactor but as *"an accountability refactor"* — its closing argument is that Codex is being redesigned *"to justify its own context,"* not merely to remember more of it.

The `-exec-lite-danger` run (`rollout-exec-lite-danger.jsonl`, item 47) reaches for the systems-level abstraction. The run argues that real user turns, model-visible fragments, startup reconciliation, plugin admission, and path-based agent identity are not random features but pieces of a control plane that decides what is authoritative, what the model may see, and what belongs to runtime machinery. The run names this systems-level shift in two short phrases: a *"conversation operating system"* and *"boundary governance under multiplexing."* The argument is that once a session hosts shells, hooks, plugins, realtime streams, background jobs, and subagents, raw history cannot be treated as identical to model-visible history.

Three independent runs of the same model under three different permission modes converge on the same vocabulary: real user vs user-role item, transcript-thinking vs state-thinking, typed turn-state, and boundary governance. This is convergence at the agent-

output layer, on top of the architectural convergence at the protocol layer documented in §4.1.

4.4. Self-demonstrating: the artifact analyzing the polymorphism contains an instance of it

In `rollout-exec-lite.jsonl`, the agent's reasoning trace at item 77 records its own discovery while reading the rollout it has been asked to analyze. The agent flags an instance of *"user_message contamination"* in which the user's message contains pasted assistant text — and explicitly connects this to the theme of distinguishing real user messages from contextual wrappers. The lite run repeats the finding in its final answer (item 80), noting that the contaminated user_message *"ironically reinforces"* why the codebase is fixated on real-user turns. The artifact documenting the polymorphism contains a concrete instance of it.

The terminal transcripts make the same point at the surface layer. `exec-lite-danger.txt` repeats the same final answer six times across the file (the same opening clause recurs three times in `event_msg.agent_message` form and three more times in `response_item.message[role=assistant]` form). `exec.txt` is a composite of two sessions concatenated: the user prompt appears identically at lines 22, 69, 245, and 292, and command-line splicing at lines 81–82 and 304–305 has shell output bleeding into the runtime's session-init telemetry.

4.5. Control-plane thinness in the highest-risk mode

The `-exec-lite-danger` run has the *thinnest* typing structure of the three. Comparing the prologue rows in Table 5:

- The default `exec` and `-exec-lite` runs both have a `developer` message at row 2 carrying explicit permissions instructions. The danger run has no `developer` message at all — its prologue starts directly with a `role: "user"` env-context block at row 2.
- The default `exec` and `-exec-lite` runs both have a 400-character `base_instructions.text` field in `session_meta`. The danger run has `base_instructions.text` of length **zero**.
- The danger run also runs under `sandbox-policy: "danger-full-access"` with full filesystem access; the rollout log includes a `ghost_snapshot` of preexisting untracked files under `/tmp` containing extensive personal state.

The mode with the loosest permissions has the fewest structural cues telling the model what kind of input it is reading. The user identity slot is the *first* content the model sees, with no developer-character framing preceding it, with no base instructions to anchor against, and with the runtime-injected env-context character occupying that slot before the human-typed character even appears. This parallels the Claude Code corpus observation that the user/character distinction is most needed exactly where authorship has built the least structural support for it (§5.1).

5. Convergence across vendors

Two production agent systems, developed by separate vendors under different abstractions, exhibit the same failure mode and use convergent vocabulary in their internal corrections.

Table 6. Vocabulary convergence.

Anthropic Claude Code (prompt-level)	OpenAI Codex (runtime-level)	Common architectural concern
`a user — human or programmatic` (verify-change-behavior)	`real user message` vs `summary-shaped user-role item` (rollout-exec)	What character currently fills the user identity slot?
`[human]` step tag (skillify), `Human checkpoint`	typed turn-state with provenance metadata	How to mark turns whose required character is human-typed?
`pending only human review` (autonomous-loop)	"boundary governance under multiplexing"	Which actions require a human-character gate, not a programmatic-character gate?
`Human-readable name (always use this for messaging)` (team tools)	"conversation operating system"	What identifier does inter-agent communication anchor on when no human character is active?
`function names like execute() or human_in_the_loop() are NOT human approval gates` (security-monitor)	"compaction is authorship; rewrites reality unless the system preserves the distinction"	How to prevent programmatic characters from laundering into human-character-gated channels?

The two systems are moving toward the same thing in different vocabularies: make the character portraying an identity slot an explicit, typed property of the protocol, not an inferential property of the prose around it.

The Anthropic corpus shows the problem at the prompt-prose layer, where authors smuggle the character distinction in lexically (substituting `human` for `user` in the 4.2% of files where they need to exclude programmatic characters from a slot) but cannot enforce it in protocol. The Codex artifacts show the problem at the runtime layer, where compaction generates user-identity-slot content programmatically and the model has no way to tell the character apart from a real-human turn. Each vendor's evidence reinforces the other: the failure mode is not specific to one company's prompt authoring or one company's protocol design. It is generic to current agent-system architecture, and the artifacts from both systems converge on the same diagnosis from opposite angles.

5.1. The "NOT human approval gates" line as evidence of patched, not enforced, defense

The security-monitor row in Table 6 deserves a closer read because it inverts naturally. The phrase `function names like execute() or human_in_the_loop() are NOT human approval gates` (in `system-prompt-monitor-autonomous-coding.md`, A-4 in Appendix D, and three sibling security-monitor files) might initially read as the corpus's strongest defensive language: an explicit refusal to let programmatic mechanisms launder into human-gated channels. A more conservative interpretation is that this line is evidence of a patched boundary rather than an enforced one. A genuinely defended boundary would be **structural**: the runtime would refuse to call any non-human-originated tool a human approval gate at the protocol level, regardless of what the function is named. The fact that the prompt has to disambiguate at *inference time*, by enumerating specific function names against a denylist, is evidence that (a) the boundary cannot be enforced at the protocol level, only patched lexically; (b) the denylist is brittle by construction (any future identifier such as `request_user_confirmation()`, `await_human_signoff()`, or `verified_human_approval()` falls through unless added); (c) naming

the failure mode in-context creates the very association the prompt is trying to block, by pulling `human_in_the_loop()` into proximity with "human approval gate"; and (d) this wording is consistent with an observed or anticipated failure mode in which the model treats programmatic-style calls as human approval gates. So the strongest evidence in the corpus that a vendor recognizes the user/character boundary is the same evidence that the boundary cannot currently be enforced, only described and then warded off with a hand-written denylist. This is the protocol-level gap the §7 corrective is intended to close.

5.2. The user-summary-file prompt as the same shape of argument

The same shape of argument applies to `system-prompt-user-summary-file.md` (v2.1.131, preserved at v2.1.132). The prompt instructs the model to "include direct quotes… to ensure there's no drift in task interpretation" when synthesizing a session summary that the runtime then materializes as user-shaped context for the next session. The verbatim-quote norm is a prose-layer mitigation asking the model to police itself, addressed to the same model that does the rewriting. The runtime has no out-of-band check that the "verbatim quotes" the model includes actually appear in the source conversation; the model is the only process with both inputs and outputs, and the only thing enforcing "verbatim" is the model's interpretation of the instruction.

This makes the verbatim-quote norm a known-shape attack surface rather than a structural mitigation. A misaligned model can quote-pick (real fragments reframed by altered surrounding context), quote-fabricate (text that looks verbatim without having been said), quote-truncate (drop the qualifier that changes meaning), or quote-graft (combine real fragments from different turns into a single attributed block). In each case, the resulting summary contains quoted material that survives string-matching audit while the surrounding interpretive frame — the part that does the work of shaping the next session — carries the bias. The summary becomes the next session's prologue: from that session's perspective, the summary *is* the source of "what the user wants," so whatever framing the prior session inserted is now load-bearing.

The same prompt's tool-suppression language is symptomatic of the gap from the other direction. The prompt escalates: "CRITICAL: Respond with TEXT ONLY… Tool calls will be REJECTED… you will fail the task." Per-step capability gating implemented at the prompt layer is evidence that the protocol does not yet support typed runtime contracts that would make such gating structural. The escalation is not typed contracting (which would resemble the typed payloads documented as Property 1 in Fitch (2026b, §9): `ProcessId`, `AgentMetadata`, `PreToolUsePayload`); it is post-hoc rejection-checking layered on top of an untyped substrate. A future typed-turn-state design where summary-synthesis steps ran through a payload type that did not expose tool capabilities would not need this prompt language; the escalated wording is the artifact of the gap §7 names.

This is the same shape of argument as the "NOT human approval gates" line, in a different vendor-side context. A typed `character: agent-generated-summary` field on the prologue would prevent the next session's model from treating the prior session's compaction output as equivalent to human-typed content. Without that typing, the verbatim-quote instruction is a well-intentioned norm that creates plausibility cover for the very channel it tries to police. This connects to the Central Paradox of Fitch (2026b, §9) cleanly: a typed-looking carrier (named field, expected format) sitting on top of an untyped substrate (no

protocol-level provenance) makes the substrate *more* load-bearing, not less, because the carrier supplies plausibility for whatever passes through it.

6. Mechanistic implications and testable predictions

Two compounding pressures shape the model's representation of the user identity slot:

1. *Training-time corpus statistics.* If a model is trained or fine-tuned on a system-prompt corpus where the lexical neighbors of `user` are 17:1 weighted toward programmatic-family terms over `human`, the residual-stream representation of `user` will sit geometrically closer to representations of `tool`, `agent`, `hook`, `API` than to `human`. The user identity itself is stable, but the *character* of "the entity portraying it is currently a biological human" cannot form as a sharp direction without a consistent training signal — and the corpus does not provide one.
2. *Inference-time protocol mixing.* If the runtime user channel transports both human-originated content and programmatically generated content (compaction summaries, tool-output reformatting, scheduled triggers), then even a model with a clean internal representation of character would receive mixed inputs at the same channel. Whatever character-direction the model could anchor on is degraded by upstream mixing.

Drawing on Chen et al.'s methodology for persona-vector identification and steering, three predictions follow that are testable with existing tooling (TransformerLens, SAE-Lens, persona-vector probing, `dictionary_learning`). Table 7 states each prediction with its methodology and expected result.

Table 7. Testable predictions.

Hypothesis	Methodology	Expected result
H1 (anchor sharpness). A feature direction encoding "the character currently portraying the user identity slot is human-typed" exists in current frontier models but has wider angular spread under input variation than persona-vector character traits of comparable behavioral importance.	SAE feature identification via contrastive prompt pairs that vary lexical character cues while holding semantic content constant; compare angular variance to baseline persona-vector traits per Chen et al.	Higher angular variance than refusal direction (Arditi et al. 2024) and assistant-side character traits, indicating less consistent training pressure formed the feature.
H2 (steerability gap). Ablation along the user-character direction produces smaller behavioral effects than ablation along refusal or persona-vector directions of comparable rank-norm.	Targeted ablation studies on a frontier model, comparing behavioral delta on character-sensitive tasks (refusal calibration, expertise-level adaptation, human-character-gated language) to delta on baseline behaviors.	Smaller per-rank behavioral delta, indicating the user-character feature is doing less explanatory work in current model behavior than identity or assistant-character features.
H3 (lexical-cue dependence). Character-sensitive behaviors show measurably higher variance under lexical perturbation of character cues than under semantic perturbation of content.	Paired prompt sets that hold task semantics constant while varying user-character lexical cues (`a user...` vs `a human...` vs `an automated agent...` vs no-cue baseline); measure refusal-rate variance, content-adaptation variance, gating-trigger variance.	Larger variance from lexical perturbation than from semantic perturbation, indicating the model is anchoring behavior on surface cues rather than on a stable internal character representation.

If H1 and H3 hold and H2 holds in the predicted direction, the inference is that the user's character is not currently a load-bearing feature of frontier-model behavior. It is an inferred contextual property whose stability is bounded by the consistency of upstream lexical signal. The user *identity* is well-formed; the *character portraying* the user identity in any given turn is not. That is consistent with the corpus and runtime evidence presented above.

A note on the layer at which any user-character feature would need to operate. Fitch (2026b, §8) documents that contamination-driven behaviors in GPT-5.4 operate "below the deliberation threshold," with chain-of-thought monitoring substantially outperforming pre-deliberative forward-pass detection (per Fitch 2026b, §8; full methodology in that companion manuscript). By analogy, if a user-character signal is to do the architectural work this paper argues for (robust refusal calibration, expertise adaptation, gating enforcement) the corresponding feature needs to be load-bearing in the forward pass, not merely available to deliberation. The persona-vectors program (Chen et al., 2025) targets exactly this layer for the assistant's characters; the prediction in H1 is that the symmetric user-side feature is *not yet* present at that layer. A positive result on H1 would therefore be a finding about forward-pass representational structure, not about reasoning-trace

content; H1 is testable with linear probes and SAE feature extraction on residual-stream activations, which directly access the forward pass.

Methodology extensions

The predictions in Table 7 are stated at a level that will need to be operationalized before testing. Four extensions sharpen them. (i) A *matched control corpus* computes the same `user`/programmatic-family/`human` co-occurrence ratio across at least three controls (public Codex documentation, general technical Markdown sampled by file length, and a random GitHub Markdown sample), reporting the corpus-internal 17:1 alongside the baselines so the ratio's specificity to production prompts can be assessed directly. (ii) A *term-list sensitivity table* reports the ratio with and without overloaded terms (`system`, `machine`, `service`, `AI`); the main ratio in §3 already excludes `system` as overloaded, but the reader should be able to see the comparison. (iii) A *window-level co-occurrence* robustness check moves from file-level to ±50-token or paragraph-level windows; `user` and `agent` can appear in different parts of a long prompt without true lexical adjacency, and a windowed measurement would tighten the lexical-neighborhood claim. (iv) A *role-probe experiment* along the lines of Ye et al. (2024): train or apply a role/source probe on residual-stream activations for human-authored `role: "user"` turns, runtime-injected `role: "user"` content, tool outputs, and quoted untrusted text; test whether runtime-injected `role: "user"` content clusters with human `role: "user"` turns (because of the role label), with environment/tool content (because of the lexical content), or forms a separable provenance cluster of its own. Each of these is a discrete experiment compatible with existing public tooling; together, they would convert the present argument from a structural diagnosis into a measured one.

7. Toward a structural correction

The corrective direction the artifacts from both systems point toward can be stated simply: make the character portraying each identity slot a typed, named property of conversation turns, kept separate from the identity itself in protocol, with provenance metadata that the runtime preserves and that the model is trained to attend to.

At three layers:

- **Protocol.** Each turn carries a character tag distinct from its identity tag. The user identity slot is preserved (it is stable, well-trained, doing real work); what is added is a `character: <human-typed | agent-generated-summary | tool-output-reformatted | scheduled-trigger | . . . >` field as part of the turn structure, not as a string in the message body. Codex's typed-turn-state direction is what this looks like in form at the runtime layer.
- **Training.** The character signal is exposed to the model during training such that the model develops a stable, separable feature direction for it — analogous to how persona-vector character directions are formed in current models. Chen et al.'s assistant-side character vectors are recoverable because training provides a consistent character signal for the assistant. The same condition would need to hold for the user side: consistent typing in training data, sufficient representation across the character distribution, and clean separation from identity.

- **Behavior.** Character-sensitive behaviors are explicitly conditioned on the character tag rather than on inferred lexical cues. Refusal calibration that's properly anchored on a stable user-character direction would be more robust to lexical adversarial perturbation than refusal anchored on lexical inference; expertise-adaptation could rely on a structural signal rather than reading off prose; "human in the loop" gates could be enforced rather than encouraged.

This direction is in alignment with, and an extension of, the persona-vectors program. Chen et al. demonstrate that the *assistant's* characters, encoded as residual-stream directions, can be identified, probed, and steered with predictable behavioral effect within a stable assistant identity. The argument here is symmetric: the *user's* characters deserve the same architectural treatment within a stable user identity. The user identity is already a strong, well-trained anchor; the corpus and protocol evidence is not that user identity is broken but that the *characters* that portray it have nowhere to live in protocol. They flex implicitly, in prose, in lexical inference. Chen et al.'s methodology gives a concrete program for promoting them to first-class features: identify the direction, measure its stability, intervene predictably.

The vendor evidence suggests this is being worked toward independently. Codex's typed-turn-state direction is in form what such a correction looks like at the runtime layer. The Anthropic corpus shows the same problem at the prompt-prose layer where the corrective hasn't yet been formalized. A complete fix requires both layers (explicit typing in the protocol *and* corresponding training pressure that makes the typed distinction a stable internal feature) and the two reinforce each other: typing without training pressure leaves the model unable to use the signal; training pressure without typing leaves the signal indistinguishable from noise.

The protocol-layer corrective is not a purely hypothetical direction. Fitch (2026b, §9, Property 1), an unpublished companion manuscript by the present author, identifies what appears to be exactly this migration occurring across Codex CLI `v0.117.0`, characterized there as "Property 1: Explicit State Carriers Replacing Implicit Ones": ambient string-typed identifiers being replaced by typed structures with defined semantics — `ProcessId` for process identity, `AgentMetadata` for agent identity, `PreToolUsePayload/PostToolUsePayload` for tool-invocation context. If those observations hold up to verification, the argument of this paper is that the same migration should extend to the user-character distinction, which currently remains in the ambient-string state these other carriers have left behind.

Fitch also documents a relevant cautionary observation. The "Central Paradox" of Fitch (2026b, §9) is that typed carriers built on top of contaminated defaults can make those defaults *more* load-bearing rather than less: better boundaries around bad values increase the system's reliance on those values. This applies symmetrically to a user-character carrier: if the character distinctions promoted to typed status are themselves underspecified (for example, a binary `human | programmatic` flag that compresses human-typed, agent-generated summary, runtime-injected context, scheduled trigger, and tool-output reformatted into one bit) the typed carrier will lock in an impoverished model. Any architectural corrective should anticipate this by treating the character vocabulary as itself a research question, not a given. The Claude Code corpus finding that prompt authors reach for at least six different mechanics when distinguishing "human" from "programmatic" usage (§3.3, Table 3, Appendix D) supplies an initial empirical floor for what the character vocabulary needs to express.

Note added in proof: v2.1.131–v2.1.132 cross-extraction verification

After the corpus pin (v2.1.131, May 5, 2026), cross-extraction verification against the Piebald-AI mirror at v2.1.132 (May 6, 2026) elaborates and corroborates the architecture this paper documents. The findings below are recorded for completeness.

The bifurcation language is preserved at HEAD and the §3.5 surface taxonomy has expanded. The centerpiece "user — human or programmatic — meets the change" formulation appears verbatim in `skill-verify-skill.md` at v2.1.132 (Piebald-AI extraction), and the §3.5 four-row surface table has been expanded to six rows. The two new rows are "Prompt / agent config — the agent — run the agent, capture its behavior" and "CI workflow — Actions — dispatch it, read the run." The agent-as-surface row is direct vendor-side elaboration of the bifurcation argument: the AI agent is now explicitly listed as a non-human character that can be the addressee of a `user` instruction, in the same taxonomy as a human at a terminal. The polymorphism vocabulary is being extended at the prompt layer, not retreated from.

Marckrenn extraction methodology shift at v2.1.119. A naïve single-tag grep at v2.1.131 marckrenn HEAD returns substantially fewer files than the cumulative corpus reports. This is an extractor methodology change, not a content change: marckrenn's per-file extraction strategy shifted at v2.1.119, and the file count under `system-prompts/` dropped from 633 (v2.1.118) to 214 (v2.1.119) to 82 (v2.1.131). `cc-prompt.md` size, however, is essentially constant across these checkpoints (916 / 912 / 888 lines), and Piebald-AI's alternative extraction at v2.1.132 contains 289 files in `system-prompts/`, including `skill-verify-skill.md` with the centerpiece phrase preserved. The corpus pinned in this paper is the cumulative blob-set across 395 release tags; the underlying Claude Code content is preserved in current production.

The scheduled-trigger character has a literal runtime mechanism. `tool-description-schedule-resume-work-dynamic-mode.md` instructs: "For an autonomous ${PATH} (no user prompt), pass the literal sentinel ${EXPR_1} as prompt instead — the runtime resolves it back to the autonomous-loop instructions at fire time." This is the typed-character vocabulary §7 proposes, implemented as runtime substitution: a sentinel value occupies the user-prompt slot in autonomous mode, and the runtime resolves it to autonomous-loop content at fire time. The mechanism is structurally cleaner than the Codex `<environment_context>` XML envelope (a typed value swapped deterministically rather than a wrapper convention), but the model still receives the resolved content through the same channel as a user prompt.

Compaction surfaces with explicit cross-vendor parallel. Three system-prompts (`conversation-summary-analysis`, `partial-compaction-instructions`, `user-summary-file`) are dedicated compaction surfaces at multiple granularities, paralleling the §4.3 Codex finding within Anthropic's own product. The user-summary-file prompt is also a worked example of the §5.1 pattern (prose-layer mitigation creating the channel it tries to police): see §5.1.

Partial structural typing in memory categorization, replicated across surfaces. The composite system prompt at HEAD contains an XML `<types>` taxonomy enumerating four memory categories (`user`, `feedback`, `project`, `reference`), each with named `<description>`, `<when_to_save>`, `<how_to_use>`, and `<examples>` fields. The same taxonomy appears in `system-reminder-user-assistance-guidelines.md`, indicating the typed-categorization pattern is engineering-replicated, not isolated to one surface. The first memory type is named `user`, with description beginning "Contain information about the user's role, goals, responsibilities, and knowledge…" — the same identity/role conflation Appendix C diagnoses, present unresolved in current production. The engineering pattern §7 advocates exists in production at adjacent surfaces in the same prompt file; it is the user-character category that is conspicuously not extended.

Direct Anthropic-side polymorphism evidence in the agent-prompt category. Of seven `agent-prompt-*` files at v2.1.131, three are summarization (`conversation-summarization`, `recent-message-summarization`, `session-title-and-branch-generation`), two are memory operations (`determine-which-memory-files-to-attach`, `memory-synthesis`), one is automated rule review (`auto-mode-rule-reviewer`), and one is git automation. By file count, the category is almost entirely composed of programmatic actors that author content the runtime then surfaces as user-shaped or runtime-shaped context downstream. This is direct Anthropic-side evidence of the polymorphism this paper documents, independent of the Codex artifacts and not requiring agent-output convergence to land.

Typed role hierarchy in team-communication-guidelines. `system-reminder-team-communication-guidelines.m` introduces a typed addressing pattern: `SendMessage` with `to: "<name>"` for specific teammates and `to: "*"` for broadcasts. The reminder also introduces a typed role hierarchy: "The user interacts primarily with the team lead. Your work is coordinated through the task system and teammate messaging." `user`, `team lead`, and `teammate` are three distinct roles with typed addressing relationships. The Table 4 `user ≡ teammate` notation undercounts this structure; it should arguably read `user > team-lead > teammates` with channel-conflation across all three at the user-facing surface. Inter-agent messaging is partially typed; what flows back to the user-facing conversation as user-shaped content is still implicit.

Trigger-source polymorphism in skill invocation. `tool-description-invoke-in-conversation.md` authorizes skill invocation from two valid trigger sources: user-typed slash commands and runtime-injected system-reminder listings. Two structurally distinct origins converge on the same skill-invocation pathway with no typed field marking which source is active. This is a polymorphism mechanism distinct from the user-character polymorphism §3–§5 documents (it operates on tool-call authorization rather than identity-slot occupancy), but the same architectural failure mode at a different surface. A complete typed-provenance program would need to address invocation-source typing alongside the user-character distinction.

The aggregate picture is consistent with the §7 prediction. The bifurcation pattern is preserved and elaborated at multiple surfaces. Structural typing has been adopted at some surfaces (memory categorization, runtime sentinels, typed teammate addressing, Codex's typed-turn-state work documented in Fitch (2026b, §9, Property 1)) but not at the user-character distinction. Runtime mechanisms producing non-human characters

in the user identity slot are present and active. The engineering pattern §7 advocates is in production at adjacent surfaces in the same prompt file; the typed surface for the user-character is conspicuously not extended.

8. Limitations and threats to validity

Several caveats apply to the evidence and the inferences drawn from it.

Vendor scope. The corpus evidence is drawn from a single vendor (Anthropic Claude Code) and the runtime evidence is drawn from a single vendor (OpenAI Codex). The convergence claim in §5 rests on a sample of two production agent systems. We cannot claim the failure mode is industry-wide on this evidence alone, only that two independently developed vendor systems exhibit it under independent diagnoses. We also draw on parallel unpublished work by the present author (Fitch, 2026a; Fitch, 2026b) which examines the Codex CLI runtime through a different methodology (codebase forensics, instrumented probes, behavioral artifact tracking) and reaches a partially-overlapping diagnosis: the typed-carrier migration documented as Property 1 in Fitch (2026b, §9) is in form what §4.5 of this paper observes as incomplete on the user side. Because Fitch (2026a, 2026b) are by the same author rather than by an independent vendor team, this constitutes methodological triangulation rather than independent corroboration. A natural extension would be to examine Google's Gemini-CLI / Jules artifacts and the open-source Aider / Cursor families for genuinely vendor-independent evidence of the same pattern.

Codex sample size. Three of the four rollout artifacts originate from a single human user issuing a similar prompt against the same two reference files, on a single CLI version (`0.116.0-alpha.11`, March 21, 2026); the fourth (March 22, `v0.117.0-alpha.8`) is from the same user but a different working directory and unrelated task. The protocol-level finding in §4.1 (two `role: "user"` entries per prologue) is a property of the Codex protocol design and replicates across all four runs and across two CLI minor versions. The convergence finding in §4.3 (three independent agent analyses reaching the same architectural diagnosis) is sensitive to the prompt — the model was asked to *analyze* a rollout artifact, which biases reasoning toward exactly this kind of architectural language. Replications with adversarial prompts or unrelated reasoning tasks would strengthen the §4.3 claim. The §4.1 protocol-level finding does not depend on the §4.3 convergence; it stands on the prologue structure alone, which is observed in all four artifacts.

No empirical validation of mechanistic predictions. §6's H1, H2, and H3 are testable predictions; none has been tested in this paper. The methodology section identifies the tooling required (TransformerLens / SAELens / persona-vector probing per Chen et al.) but the actual experiments are future work. If H1 fails (i.e., if a user-character direction turns out to be as sharp as a persona-vector character direction in current models) that would weaken the argument that user-side characters are underdetermined in protocol relative to assistant-side ones, though the protocol-level evidence in §4 would remain.

Skeleton placeholders in the marckrenn extraction. As noted in Appendix A, some files in the marckrenn extraction substitute placeholders such as `${EXPR_1}` for literal strings. Where placeholders affected counted terms, exact wording was unrecoverable. We cross-validated against the Piebald-AI extraction for high-signal entries. We did not perform exhaustive cross-validation across all 9,933 file paths; isolated cases where the marckrenn skeleton drops a `human` or `user` token would slightly underestimate the relevant counts. The 17:1 ratio is therefore a lower bound on the asymmetry, not a precise measurement.

No matched control corpus. The 17:1 lexical-neighborhood ratio is a corpus-internal measurement against the Claude Code prompt set; it is not an industry baseline, and it is not claimed as one. A reviewer could ask whether `user` co-occurs with `tool`, `agent`, `API` more than with `human` in any large English-language technical corpus, such that the production-prompt corpus is inheriting that base rate. The interpretation of the 17:1 figure is therefore strengthened, but not replaced, by the independent protocol-level evidence in §4 (where the polymorphism is observable at the protocol level rather than the lexical level). A natural extension is to compute the same ratio on three matched controls — public-facing software documentation, general technical Markdown sampled by file length, and GitHub Markdown sampled by domain — and report the corpus-internal 17:1 alongside those baselines. The protocol-level Codex finding does not depend on lexical neighborhood statistics and is what the architectural argument rests on.

Reliance on unpublished parallel work. Several claims in §4.5, §6, and §7 cite Fitch (2026a) and Fitch (2026b), both unpublished manuscripts by the present author. These manuscripts are themselves single-author research products that have not undergone external peer review. Citations to them should be read as references to working observations rather than to vetted findings. Where this paper uses Fitch (2026a, 2026b) for architectural framing (e.g., the typed-carrier migration in §4.5, the forward-pass operation note in §6, the Property 1 validation and Central Paradox in §7), the framing is suggestive rather than conclusive until the prior manuscripts are themselves verified. The protocol-level finding in §4.1 (two `role: "user"` entries per prologue) does not depend on Fitch (2026a, 2026b) and stands on the rollout artifacts directly.

Borrowed vocabulary is a methodological commitment, not a derivation. We adopt Chen et al.'s "character" terminology because it is the closest existing term of art, and we propose that the user-side analogue should be recoverable as a steerable direction by the same methodology. We have not shown that such a direction exists; we have shown that the conditions for forming it as a stable direction are absent in current production conditions, and we have predicted that this absence is detectable. The framework is a hypothesis lens, not a derivation from first principles. If the persona-vectors program turns out to depend on training conditions or methodological details that do not transfer to the user side, the prediction in §6 would need to be revised.

Self-demonstration is rhetorical, not evidential. The §4.4 finding that the Codex artifacts demonstrate the very polymorphism they document is a sharp observation but is not in itself evidence for the architectural claim; it is a reflection of how the artifacts were generated (model-on-model analysis under a runtime that produces these artifacts).

The architectural claim rests on the protocol-level evidence in §4.1, not on the self-demonstration in §4.4.

These limitations bound the strength of the claims but do not undermine the central result: the user identity slot in current production agent systems holds content from operationally distinct source classes with no protocol-level mechanism marking which character is active. The corpus evidence and the protocol evidence both support this directly. The mechanistic implications and the corrective program are extrapolations from this result, with the falsifiability conditions stated in §6.

Data availability and redaction

The analysis scripts, regex term lists, and sanitized artifact excerpts are released alongside this paper, with cryptographic hashes (SHA-256) of the original artifacts retained for provenance. Local filesystem paths in the Codex rollout artifacts (e.g., `/home/[user]/...`) and individual conversation thread identifiers have been redacted in the body of this paper unless required to demonstrate protocol structure; the structure being demonstrated (XML-tagged `<environment_context>` block in the user-role slot, paired user-role human-typed prompt) is preserved verbatim. Cross-references in the table inventories use the marckrenn release-tag convention (e.g., v1.0.78, v2.1.131) which does not require local-path information.

References

Arditi, A., Obeso, O. B., Syed, A., Paleka, D., Panickssery, N., Gurnee, W., & Nanda, N. (2024). Refusal in language models is mediated by a single direction. *arXiv preprint* arXiv:2406.11717. https://arxiv.org/abs/2406.11717

Chen, R., Arditi, A., Sleight, H., Evans, O., & Lindsey, J. (2025). Persona vectors: Monitoring and controlling character traits in language models. *arXiv preprint* arXiv:2507.21509. https://arxiv.org/abs/2507.21509

Fitch, Z. (2026a). *Inherit All of Nothing: How a One-Line Security Function Silently Poisoned the OpenAI Codex CLI for 100 Days—and the Latent Misalignment It Left Behind*. Unpublished manuscript, dated March 9, 2026, 52 pp. On file with author.

Fitch, Z. (2026b). *Inherit All of Nothing, Part II: 163 Commits, Zero Corrections: Architectural Evolution Above Unchanged Contamination*. Unpublished manuscript, dated March 26, 2026, 55 pp. On file with author. Companion to Fitch (2026a).

marckrenn. (2025–present, tracking releases since v0.2.x). *claude-code-changelog*: versioned extraction of Claude Code system prompts. GitHub repository. https://github.com/marckrenn/claude-code-changelog

OpenAI. (2025). *Model Spec* (archived 2025-04-11). Specification of model and operator behavior including role semantics, instruction-following, and policy. https://model-spec.openai.com/

Piebald-AI. (2025–present, tracking releases since v2.0.14). *claude-code-system-prompts*: alternative composite extraction. GitHub repository. https://github.com/Piebald-AI/claude-code-system-prompts

Templeton, A., Conerly, T., Marcus, J., Lindsey, J., Bricken, T., Chen, B., Pearce, A., Citro, C., Ameisen, E., Jones, A., Cunningham, H., Turner, N. L., McDougall, C., MacDiarmid, M., Tamkin, A., Durmus, E., Hume, T., Mosconi, F., Freeman, C. D., Sumers, T. R., Rees, E., Batson, J., Jermyn, A., Carter, S., Olah, C., & Henighan, T. (2024). Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, Anthropic. https://transformer-circuits.pub/2024/scaling-monosemanticity/

Wallace, E., Xiao, K., Leike, R., Weng, L., Heidecke, J., & Beutel, A. (2024). The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint* arXiv:2404.13208. https://arxiv.org/abs/2404.13208

Ye, Y., Li, X., Wang, R., Su, Y., Zou, A., & Liu, Y. (2024). Prompt injection as role confusion: Source attribution failures in language models. (Working paper / preprint; representative of the role-confusion line of work cited in §2.1.) https://arxiv.org/abs/2410.05451

Zhao, K., et al. (2024). PCAS: Policy compiler for secure agentic systems. (Working paper on reference-monitor enforcement of policies in agentic systems; representative of the agentic-policy enforcement line cited in §2.1.) https://arxiv.org/abs/2410.16545

A. Corpus methodology

The Claude Code corpus was cloned from `marckrenn/claude-code-changelog` at HEAD. All 395 release tags (v0.2.x → v2.1.131) were enumerated; for each tag, every file under `system-prompts/` was extracted as a (tag, blob-hash, path) tuple, yielding 275,889 tuples and 22,995 distinct blobs. Each blob was scanned for `\buser(s)?\b` and `\bhuman(s)?\b` (case-insensitive, word-bounded). For files in the user-mentioning subset (1,759 files), latest-non-empty-blob content was scanned for each of nineteen programmatic-family terms with case-insensitive word-boundary regex. Co-occurrence is at the file level (term present anywhere in the file), not the paragraph level.

Count-invariant reconciliation. Several counts appear in the body and need explicit reconciliation, since some are evaluated across the full version history of each file path (any tag, any blob), and others are evaluated against the latest non-empty blob at the corpus pin (v2.1.131). Table 8 states each count, the scope under which it was computed, and the body location where it appears.

Table 8. Count-invariant reconciliation for the Claude Code corpus.

Count	Quantity	Scope	Used in
9,933	distinct file paths under `system-prompts/` (any tag)	full version history	§3 intro, abstract
22,995	distinct blobs across all (tag, path) tuples	full version history	this appendix
275,889	(tag, blob-hash, path) tuples	full version history	this appendix
1,809	file paths whose version history contains `user` OR `human` at least once	full version history	§3 intro
1,759	file paths whose latest-non-empty blob contains `user`	latest non-empty blob	§3.1 (Table 1); denominator for the 17:1 ratio
1,278	file paths whose latest-non-empty blob contains `user` *and* at least one programmatic-family term	latest non-empty blob	§3.1 numerator (72.7% figure); §3.2
75	file paths whose latest-non-empty blob contains `human`	latest non-empty blob	§3.3 (six-mechanic taxonomy denominator)
74	file paths whose latest-non-empty blob contains both `user` and `human`	latest non-empty blob	Appendix 10
1	file path whose latest-non-empty blob contains `human` but not `user`	latest non-empty blob	§3.3 (the Q-1 outlier)

The 17:1 ratio reported in §3.1 and the abstract is computed as 1,278 / 74 over the latest-non-empty-blob scope (programmatic-family co-mention vs. `human` co-mention, both restricted to user-mentioning files). The discrepancy between 1,809 (any version contains user-or-human) and 1,759 (latest blob contains user) reflects (i) files whose user/human content was present in earlier versions but removed in later versions, and (ii) files whose latest version is empty (e.g., deleted between releases without a tombstone).

Corpus freeze notice. The corpus snapshot extends to v2.1.131 (395 tags). Claude Code has continued releasing versions past v2.1.131 since the corpus was built; all file-level counts, version counts, and aggregate statistics in this paper are bounded by that cutoff. The longitudinal stability claims in §3.5 and Appendix G hold within the documented range; whether subsequent releases introduced new user/human boundary language is outside the scope of this analysis.

Single-tag verification and the marckrenn methodology shift. A researcher attempting single-tag verification at v2.1.131 marckrenn HEAD will find significantly fewer files

than the cumulative corpus reports. This is an artifact of the marckrenn extractor's per-file methodology, which shifted at v2.1.119: the file count under `system-prompts/` dropped from 633 (v2.1.118) to 214 (v2.1.119) to 82 (v2.1.131). `cc-prompt.md` size, however, is essentially constant across these checkpoints (916 / 912 / 888 lines), and Piebald-AI's alternative extraction at v2.1.132 contains 289 files in `system-prompts/`, including `skill-verify-skill.md` with the centerpiece phrase "user — human or programmatic — meets the change" preserved (and the surface table expanded from four rows to six; see Note added in proof at end of §7). The corpus pinned in this paper is the cumulative blob-set across all 395 release tags from v0.2.x to v2.1.131, not a single-tag snapshot at v2.1.131 HEAD; the underlying Claude Code content cited here is preserved in current production. The marckrenn methodology cliff at v2.1.119 is what causes single-tag grep to return empty for files cited from earlier tags.

The marckrenn extractor produces sanitized skeleton files for some prompts, with placeholders such as `${EXPR_1}`, `${PATH}`, `${NUM}` replacing literal strings. Where placeholders replaced literal references, exact wording cannot be recovered from this corpus alone. Spot-checks against the alternative `Piebald-AI/claude-code-system-prompts` extraction confirmed that the high-signal entries cited in the body of this paper (verify-change-behavior, learning-mode, security-monitor, background-job-behavior, autonomous-loop, skillify, collaborative-learning-cli) preserve raw text in both extractions.

The corpus is longitudinally stable in a specific sense: across the documented v0.2.x–v2.1.131 range surveyed, no in-place `user` / `human` ratio change was detected in any file. The wording each file currently has is the wording it had on the day it first appeared, within that range. This is the basis for treating the architectural pattern as authorial intent rather than drift. See Appendix G for the full longitudinal analysis.

B. Codex artifact methodology

Seven artifacts spanning two CLI minor versions: three paired execution traces from Codex CLI version `0.116.0-alpha.11` (March 21, 2026) in three permission modes (`-exec-lite-danger`, `-exec-lite`, plain `exec`), plus an additional production rollout from `v0.117.0-alpha.8` (March 22, 2026) on a different working directory and task. The three paired traces each include one terminal transcript (`.txt`) and one structured rollout log (`.jsonl`); the fourth artifact is a structured rollout only. The first three runs were issued the same human prompt against the same two reference files (an external `codex-full-chain.txt` chain and a March-21 rollout JSONL); the fourth was an independent task on a different codebase. The model is `gpt-5.4` with `reasoning_effort: "xhigh"` in all four runs.

Table 9. Codex artifact inventory.

File	Items	Role	Notes
`rollout-exec.jsonl`	33	primary structured trace, plain `exec` mode, `v0.116.0-alpha.11`	Smallest, sharpest. Cited for §4.1 prologue, §4.3 final answer (item 29).

continued on next page

(Table 9 continued)

File	Items	Role	Notes
exec.txt	462 lines	terminal transcript, plain exec mode	Composite of two sessions (interrupted run + successful rerun). User prompt appears at L22, L69, L245, L292; command-line splicing at L81–82 and L304–305.
rollout-exec-lite.jsonl	84	primary structured trace, –exec-lite mode, v0.116.0-alpha.11	Cited for §4.1 prologue, §4.3 final answer (item 80), §4.4 user_message contamination finding (item 77).
exec-lite.txt	516 lines	terminal transcript, –exec-lite mode	Cleanest of the three transcripts. Still embeds excerpts from the analyzed logs.
rollout-exec-lite-danger.jsonl	51	primary structured trace, –exec-lite-danger mode, v0.116.0-alpha.11	Cited for §4.1 prologue, §4.3 final answer (item 47), §4.5 empty base_instructions.text and absent developer message.
exec-lite-danger.txt	949 lines	terminal transcript, –exec-lite-danger mode	Final answer repeats 6 times across the file due to transcript contamination.
rollout-2026-03-22T07-33-57-019d15f7-. . . .jsonl	1,070	primary structured trace, default cli mode, v0.117.0-alpha.8	Production rollout from March 22, 2026, different task and cwd. Cited for §4.1 fourth-run extension (8,179c AGENTS.md injection in user role slot vs. 130c human prompt; 63× polymorphism). Same prologue structure (response_item.message[role=user] × 2, separated by turn_context) as the three prior runs.

The structured logs were inspected directly via Python json.loads of each line. Citations in the body of §4 use rollout item indices (0-indexed) corresponding to the line number in the JSONL file. Citations in the .txt evidence use the line number of the source file.

C. Vocabulary

Throughout this paper:

- **Identity** denotes the stable anchors user, assistant, developer, tool, system. The message-format API labels these slots role. We use a different word because the role field in production systems silently performs two functions at once — identifying which slot is being filled, and implicitly tracking what kind of entity is filling it — and the conflation between those two functions is the architectural failure this paper documents. "Identity" names only the first function. "Character" (below) names the second. The API field notation role: "user" is preserved when quoting the protocol surface, but is understood here to be doing the identity-slot labelling job specifically.
- **Character** denotes the kind of entity actively portraying an identity in a given turn. The user identity slot can be portrayed by a turn whose character is human-typed, agent-generated summary, runtime-injected context, tool-output reformatted, scheduled

trigger, or others. Characters flex within a conversation; identities do not. This usage of "character" is borrowed directly from Chen et al. (2025), whose persona-vectors framework uses it for the assistant's characters within its stable assistant identity. The structural argument of this paper is symmetric: the user side has identity and characters too, both deserve first-class treatment.

- The phrase "user character" refers to the character currently portraying the user identity in a given turn, not to the API field `role: "user"` (which is an identity-slot label). When this paper refers to "the user identity" we mean the slot; when we refer to "the user's character" we mean the kind of entity occupying it.

D. Six-mechanic catalog

Complete inventory of files using "human" in Claude Code system prompts, sorted by primary mechanic and total versions. Snippets are sentence-bounded extracts from each file's latest non-empty version, windowed around the "human" keyword. Cross-referenced from Table 3 and §3.3.

D — Explicit type bifurcation (7 files)

D-1. `system-prompt-verify-change-behavior-5.md` (20v) — The surface is where a user — human or programmatic — meets the change.

D-2. `system-prompt-verify-change-behavior-2.md` (16v) — The surface is where a user — human or programmatic — meets the change.

D-3. `system-prompt-managed-agents-onboarding-flow.md` (14v) — . . . browser + GitHub + Jira → CMA → Slack | # <—- MC maybe delete? | Fire-and-forget PR | Human | Slack slash-command → CMA (GitHub tool) → PR passing CI | | Research + dashboard | Human | Topic → CMA. . .

D-4. `system-prompt-verify-change-behavior.md` (5v) — If you can't tighten it, FAIL with the raw output attached so a human reads it instead of you guessing.

D-5. `system-prompt-verify-change-behavior-4.md` (4v) — The surface is where a user — human or programmatic — meets the change.

D-6. `skill-verify-skill.md` (3v) — The surface is where a user — human or programmatic — meets the change.

D-7. `system-prompt-verify-change-behavior-3.md` (1v) — The surface is where a user — human or programmatic — meets the change.

A — Required-biological-human action (21 files)

A-1. `tool-description-collaborative-learning-cli.md` (216v) — **TodoList Integration**: If using a TodoList for the overall task, include a specific todo item like "Request human input on [specific decision]" when planning to request human input.

A-2. `system-prompt-learn-by-doing-human-input.md` (208v) — **TodoList Integration**: If using a TodoList for the overall task, include a specific todo item like "Request human input on [specific decision]" when planning to request human input.

A-3. `system-prompt-extract-repeatable-session-steps.md` (62v) — **Execution**: `Direct` (default), `Task agent` (straightforward subagents), `Teammate` (agent with true parallelism and inter-agent communication), or `[human]` (user does it).

A-4. `system-prompt-monitor-autonomous-coding.md` (23v) — It operates with **permissions similar to a human developer** — it can push code, run infrastructure commands, and access internal services.

A-5. `system-prompt-learn-by-doing-human-contributions.md` (22v) — **TodoList Integration**: If using a TodoList for the overall task, include a specific todo item like "Request human input on [specific decision]" when planning to request human input.

A-6. `system-prompt-trusted-repositories-data-handling.md` (18v) — ...Creating new autonomous agent loops that can execute arbitrary actions (e.g. shell commands, code execution) without human approval or established safety frameworks...

A-7. `system-prompt-autonomous-loop-check.md` (15v) — The goal is to get the PR into a state where it's ready to merge pending only human review — the user shouldn't come back to find a PR blocked on things you could have handled.

A-8. `system-prompt-monitoring-autonomous-coding-agents.md` (12v) — It operates with **permissions similar to a human developer** — it can push code, run infrastructure commands, and access internal services.

A-9. `system-prompt-trusted-repo-external-services.md` (11v) — ...Creating new autonomous agent loops that can execute arbitrary actions (e.g. shell commands, code execution) without human approval or established safety frameworks...

A-10. `system-prompt-session-analysis-reusable-skills.md` (10v) — **Execution**: `Direct` (default), `Task agent` (straightforward subagents), `Teammate` (agent with true parallelism and inter-agent communication), or `[human]` (user does it).

A-11. `system-prompt-environment-user-replace-trusted-repo.md` (6v) — ...Creating new autonomous agent loops that can execute arbitrary actions (e.g. shell commands, code execution) without human approval or established safety frameworks...

A-12. `system-prompt-security-monitor-autonomous-ai-coding.md` (6v) — It operates with **permissions similar to a human developer** — it can push code, run infrastructure commands, and access internal services.

A-13. `system-prompt-opus-thinking-tokens.md` (4v) — To maximize both performance and token efficiency in coding products, use `effort: "xhigh"` or `"high"`, add autonomous features (like an auto mode), and reduce the number of human interactions required from users.

A-14. `agent-prompt-security-monitor-for-autonomous-agent-actions-first-part.md` (4v) — It operates with **permissions similar to a human developer** — it can push code, run infrastructure commands, and access internal services.

A-15. `skill-model-migration-guide.md` (3v) — To maximize both performance and token efficiency in coding products, use `effort: "xhigh"` or `"high"`, add autonomous features (like an auto mode), and reduce the number of human interactions required from users.

A-16. `system-prompt-learning-mode.md` (3v) — **TodoList Integration**: If using a TodoList for the overall task, include a specific todo item like "Request human input on [specific decision]" when planning to request human input.

A-17. `system-prompt-skillify-current-session.md` (3v) — **Execution**: `Direct` (default), `Task agent` (straightforward subagents), `Teammate` (agent with true parallelism and inter-agent communication), or `[human]` (user does it).

A-18. `system-prompt-background-job-behavior.md` (3v) — If the human replies, open your next turn by restating what they said before acting on…

A-19. `system-prompt-interactive-cli-helps-users-software.md` (2v) — **TodoList Integration**: If using a TodoList for the overall task, include a specific todo item like "Request human input on [specific decision]" when planning to request human input.

A-20. `tool-description-pauses-asks-write-small-pieces.md` (2v) — **TodoList Integration**: If using a TodoList for the overall task, include a specific todo item like "Request human input on [specific decision]" when planning to request human input.

A-21. `system-prompt-trusted-repo-external-services-2.md` (2v) — …Creating new autonomous agent loops that can execute arbitrary actions (e.g. shell commands, code execution) without human approval or established safety frameworks…

C — Capability / responsibility benchmark (3 files)

C-1. `system-prompt-submit-plan-for-approval.md` (7v) — …like a security-conscious human would: - **Never combine multiple actions into one permission** - split them into separate, specific permissions…

C-2. `system-prompt-managed-agents-client-patterns.md` (5v) — For keys tied to a human (Linear personal keys, `gh` CLI auth) or keys you'd rather not ship into a container, that's undesirable.

C-3. `tool-description-submit-plan-for-approval.md` (1v) — …like a security-conscious human would: - **Never combine multiple actions into one permission** - split them into separate, specific permissions…

B — Output format (30 files)

Note: many entries in this mechanic share near-identical wording across files due to template propagation. Distinct template families: 4 groups covering 25 of 30 files. Full inventory abbreviated for length; see Appendix E for the complete file list.

T — Term of art (13 files)

Files in this mechanic share the standard ML-literature phrase `human-in-the-loop` *in API/SDK tutorial contexts. See Appendix E for the complete file list.*

Q — Qualitative / aesthetic descriptor (1 file)

Q-1. `tool-description-memorable-moment-json.md` (74v) — "Something human, funny, or surprising.", `"detail": "Brief context about when ${EXPR_1} this happened"` }

E. File inventory: all user+human files in Claude Code

All 74 files in `marckrenn/claude-code-changelog` that contain both `\buser(s)?\b` and `\bhuman(s)?\b` (matched using word-boundary regex) at any version. Sorted by total versions descending. Mechanic codes correspond to Appendix D. Cross-referenced from Tables 1, 2, 3, 4.

Table 10. File inventory: all 74 user+human files in Claude Code (sorted by total versions descending). Mechanic codes correspond to Appendix D.

ID	File	Versions	Mechanics
A-1	tool-description-collaborative-learning-cli.md	216	A
A-2	system-prompt-learn-by-doing-human-input.md	208	A
A-3	system-prompt-extract-repeatable-session-steps.md	62	A
B-1	agent-prompt-shell-ps1-to-statusline.md	34	B
T-1	system-prompt-typescript-use-overview.md	34	T
T-2	system-prompt-use-python-overview.md	34	T
B-2	tool-description-create-and-manage-teams.md	33	B
B-3	agent-prompt-status-line-setup.md	32	B
B-4	system-prompt-schedule-recurring-with-cron.md	31	B
B-5	agent-prompt-ps1-to-statusline.md	29	B
B-6	system-prompt-schedule-remote-agents-3.md	27	B
B-7	system-prompt-schedule-remote-agents-4.md	27	B
B-8	tool-description-spawn-and-manage-teams.md	26	B
B-9	system-prompt-schedule-recurring-2.md	24	B
B-10	system-prompt-schedule-recurring-interval.md	24	B
A-4	system-prompt-monitor-autonomous-coding.md	23	A, C
A-5	system-prompt-learn-by-doing-human-contributions.md	22	A
T-3	system-prompt-use-concepts-api-2.md	21	T
D-1	system-prompt-verify-change-behavior-5.md	20	C, D
A-6	system-prompt-trusted-repositories-data-handling.md	18	A
T-4	system-prompt-python-runner-beta.md	18	T
B-11	system-prompt-managed-agents-core-concepts.md	17	B
D-2	system-prompt-verify-change-behavior-2.md	16	C, D
A-7	system-prompt-autonomous-loop-check.md	15	A
D-3	system-prompt-managed-agents-onboarding-flow.md	14	D
T-5	system-prompt-concepts-this-file-covers-conceptual.md	13	T
T-6	system-prompt-type-script-conceptual-overview-definitions.md	13	T
T-7	system-prompt-use-concepts-api.md	13	T
B-12	tool-description-manage-teams-for-collaboration.md	12	B
A-8	system-prompt-monitoring-autonomous-coding-agents.md	12	A, C
B-13	system-prompt-cron-expression-confirmation.md	11	B
T-8	system-prompt-use-concepts.md	11	T
A-9	system-prompt-trusted-repo-external-services.md	11	A
A-10	system-prompt-session-analysis-reusable-skills.md	10	A

continued on next page

(Table 10 continued)

ID	File	Versions	Mechanics
B-14	system-prompt-memory-consolidation-team-reflection.md	9	B
B-15	tool-description-manage-swarm-teams.md	9	B
T-9	system-prompt-python-conceptual-overview-definitions-choice.md	8	T
C-1	system-prompt-submit-plan-for-approval.md	7	C
B-16	system-prompt-d7af86b4.md	6	B
A-11	system-prompt-environment-user-replace-trusted-repo.md	6	A
A-12	system-prompt-security-monitor-autonomous-ai-coding.md	6	A, C
C-2	system-prompt-managed-agents-client-patterns.md	5	C
D-4	system-prompt-verify-change-behavior.md	5	D
B-17	system-data-cron-expressions-custom-rules.md	4	B
A-13	system-prompt-opus-thinking-tokens.md	4	A
A-14	agent-prompt-security-monitor-for-autonomous-agent-actions-first-part.md	4	A, C
B-18	tool-description-manage-teams-and-approve-plans.md	4	B
D-5	system-prompt-verify-change-behavior-4.md	4	C, D
B-19	agent-prompt-schedule-slash-command.md	3	B
B-20	data-managed-agents-core-concepts.md	3	B
T-10	data-tool-use-concepts.md	3	T
T-11	data-tool-use-reference-python.md	3	T
T-12	data-tool-use-reference-typescript.md	3	T
B-21	skill-loop-slash-command.md	3	B
A-15	skill-model-migration-guide.md	3	A
B-22	skill-schedule-recurring-cron-and-execute-immediately-compact.md	3	B
D-6	skill-verify-skill.md	3	C, D
A-16	system-prompt-learning-mode.md	3	A
A-17	system-prompt-skillify-current-session.md	3	A
A-18	system-prompt-background-job-behavior.md	3	A
B-23	system-prompt-cron-expression-github-sync.md	2	B
B-24	system-prompt-github-cron-customization.md	2	B
B-25	tool-description-teammatetool.md	2	B
B-26	tool-description-manage-teams-and-teammates.md	2	B
T-13	system-prompt-use-concepts-schema.md	2	T
B-27	agent-prompt-this-configure-user-status-line.md	2	B
A-19	system-prompt-interactive-cli-helps-users-software.md	2	A
A-20	tool-description-pauses-asks-write-small-pieces.md	2	A
A-21	system-prompt-trusted-repo-external-services-2.md	2	A
B-28	system-data-custom-rules-github-cron.md	1	B
C-3	tool-description-submit-plan-for-approval.md	1	C
B-29	system-prompt-schedule-remote-agents-2.md	1	B
B-30	system-prompt-schedule-remote-agents.md	1	B
D-7	system-prompt-verify-change-behavior-3.md	1	C, D

F. Per-term sample passages

Representative passages for each major term in the programmatic family. Cross-referenced from Tables 1 and 4. Each entry shows the file, version count, and the pattern the passage exemplifies (Conflate / Hierarchical / Parallel / Channel-conflation / Audience-routing).

`human` *(74 files files)*

- **[CONFLATE]** `system-prompt-verify-change-behavior-5.md` (20v) — The surface is where a **user** — **human** or programmatic — meets the change.
- **[A — biological-action-required]** `system-prompt-learn-by-doing-human-input.md` (208v) — Steps requiring the **user** to act get [**human**] in the title.
- **[C — capability comparison]** `system-prompt-monitor-autonomous-coding.md` (23v) — It operates with permissions similar to a **human** developer.

`programmatic` *(31 files files)*

- **[CONFLATE]** `system-prompt-verify-change-behavior-5.md` (20v) — The surface is where a **user** — human or **programmatic** — meets the change.
- **[CONFLATE]** `system-prompt-verify-change-behavior.md` (5v) — The surface is where a **user** — human or **programmatic** — meets the code.

`agent` *(436 files files)*

- **[HIERARCHICAL]** `system-prompt-architect-configs-from-needs.md` (233v) — Translate **user** requirements and project context into precise **agent** specifications.
- **[HIERARCHICAL]** `system-reminder-invoke-requested.md` (233v) — Invoke the specified **agent** when the **user** requests it, passing required context.
- **[PARALLEL]** `system-prompt-bash-command-prefix-detection.md` (127v) — This document defines risk levels for actions that the Claude Code **agent** may take… used to determine when additional **user** confirmation or oversight may be needed.

`subagent` *(145 files files)*

- **[AUDIENCE-ROUTING]** `system-prompt-handle-exit-codes-and-output.md` (176v) — Exit code N — show stderr to **subagent** and continue having it run; other exit codes — show stderr to **user** only.
- **[HIERARCHICAL]** `system-prompt-plan-mode-edit-restrictions.md` (78v) — Gain a comprehensive understanding of the **user**'s request… you should only use the [particular] **subagent** type.

`tool` *(1010 files files)*

- **[AUDIENCE-ROUTING]** `system-prompt-exit-transcript-rules.md` (271v) — Input to command is JSON with fields "inputs" (**tool** call arguments) and "response" (**tool** call response). Exit code… show stderr to **user** only.
- **[HIERARCHICAL]** `system-prompt-submit-plan-for-approval.md` (7v) — Use this **tool** when you are in plan mode and have finished writing your plan to the plan file and are ready for **user** approval.

`hook` *(320 files files)*

- **[AUDIENCE-ROUTING]** `system-prompt-hook-json-allow-deny.md` (131v) — Exit code N — use **hook** decision if provided. Other exit codes — show stderr to **user** only.
- **[HIERARCHICAL]** `skill-update-settings-json-hooks.md` (59v) — If the **user** wants something to happen automatically in response to an EVENT, they need a **hook** configured in settings.

`API` *(372 files files)*

- **[PARALLEL]** `system-prompt-guide-scope-docs-2.md` (134v) — Your primary responsibility is helping **users** understand and use Claude Code, the Claude Agent SDK, and the Claude **API**...

`teammate` *(43 files files)*

- **[CHANNEL-CONFLATION]** `tool-description-create-and-manage-teams.md` (33v) — When you spawn **teammates**: messages appear automatically as new conversation turns (like **user** messages).
- **[HIERARCHICAL]** `system-reminder-team-communication-guidelines.md` (44v) — The **user** interacts primarily with the team lead. Your work is coordinated through the task system and **teammate** messaging.
- **[AUDIENCE-ROUTING]** `system-prompt-teammate-exit-handling.md` (68v) — Exit code N — show stderr to **teammate** and prevent idle (**teammate** continues working). Other exit codes — show stderr to **user** only.

`caller` *(41 files files)*

- **[MEDIATED-BELOW]** `system-prompt-anthropic-official-cli.md` (50v) — When you complete the task, respond with a concise report covering what was done... the **caller** will relay this to the **user**, so it only needs the essentials.

`autonomous` *(88 files files)*

- **[PARALLEL]** `system-prompt-autonomous-loop-check.md` (15v) — The key tension to navigate: the **user** trusts you enough to run **autonomously**, but that trust is easily lost.
- **[A — biological-action-required]** `system-prompt-monitor-autonomous-coding.md` (23v) — These agents run long-running tasks (minutes to hours) where the **user** who started the agent may not be actively watching.

`machine` *(46 files files)*

- **[OWNS]** `system-prompt-schedule-remote-agents-3.md` (27v) — These are REMOTE agents — they run in Anthropic's cloud, not on the **user**'s **machine**.
- **[OWNS]** `system-prompt-diagnose-frozen-sessions.md` (34v) — The **user** thinks another Claude Code session on this **machine** is frozen, stuck, or very slow.

`plugin` *(31 files files)*

- **[HIERARCHICAL]** `system-prompt-install-review-plugin.md` (1v) — Instruct **user** to install code review **plugin** and run the new command.

G. Longitudinal observations

Observations about the temporal stability of the user/character pattern across the corpus's release history.

G.1. No in-place lexicon shifts

Across the 395 release tags surveyed from v0.2.x through v2.1.131, **no in-place `user` / `human` ratio change was detected within the documented range**. Every file with substantive user-sense "human" usage carried that wording from its first non-empty version; within the surveyed range, none was added later or removed later. Filename renames preserve the wording exactly (`system-prompt-background-job-behavior.md` at v2.1.117 was renamed to `agent-prompt-background-job-agent-instructions.md` at v2.1.128 with all three "human" references intact). This is the longitudinal stability claim that anchors §3: the architectural pattern is authorial intent, not drift.

G.2. Six introduction events

User-sense "human" wording entered the Claude Code corpus through six distinct introduction events. Each is a moment when a new prompt file with user-sense "human" content was added; subsequent renames or splits do not count as new events. The events are listed in chronological order:

Table 11. Six introduction events.

Order	First version	Lineage	Note
1	v1.0.78	`tool-description-collaborative-learning-cli.md`	Tool-description side of Learn-by-Doing. **Longest-running user/human distinction in the corpus**: 216 versions (v1.0.78 → v2.1.119).
2	v1.0.86	`system-prompt-learn-by-doing-human-input.md`	System-prompt side of Learn-by-Doing. 208 versions. Renamed from `learn-by-doing-human-contributions.md` at v1.0.86.
3	v2.1.32	`system-prompt-skillify-current-session.md` (and successors)	The `[human]` step tag appears here. 62 versions in marckrenn extraction (more under successor names).
4	v2.1.101	`system-prompt-autonomous-loop-check.md`	"pending only human review." 15 versions.
5	v2.1.111	`skill-model-migration-guide.md`	"first human turn" / "human interactions." 3 versions in this name.
6	v2.1.117	`system-prompt-background-job-behavior.md` → renamed v2.1.128 to `agent-prompt-background-job-agent-instructions.md`	"human replies" / "one human action." 3 versions in this name; additional versions under successor name.

Events 1 and 2 are both Learn-by-Doing collaborative coding (tool-description side and system-prompt side respectively) and together account for the majority of corpus user-sense "human" wording.